Final Technical Improvements: Three Fixes

Local-only, no EC2, no re-collection. Base dataset: `tpch_dataset_corrected.csv` (unmodified, kept for provenance). Same local PostgreSQL 15.17 SF1 instance used for the buffer-bug correction and Addition 4b (`results/buffer_bug_correction_report.md`, `results/addition4b_mechanism_report.md`), re-verified here: 6,001,215 `lineitem` rows (exact match to the collection guide), same 21 patched query files, same index set, `random_page_cost=4`, `seq_page_cost=1`, `effective_cache_size=4GB`, `shared_buffers=128MB`, `max_parallel_workers_per_gather=2` — all still at the documented values, no residual mismatch found. `random_state=42` everywhere randomness is used; the parallelism timing experiment has no randomness to seed (it's a direct wall-clock measurement).

Fix 1: correcting the `rows` feature (rows processed, not rows returned)

`scripts/collection/extract_plan_features.py` runs `EXPLAIN (ANALYZE, VERBOSE, FORMAT JSON)` for all 21 queries and sums `Actual Rows x Actual Loops` across every node in the plan tree (loops-multiplication matters because `Actual Rows` is a per-loop average, not a cumulative field — unlike the buffer-bug fields, so no double-counting risk here). Written to `results/plan_structure_features.csv`.

The effect is exactly as hypothesized: `q1` (aggregate over all of `lineitem`) has `root_actual_rows=4` but `rows_processed=5,927,485`; `q18` goes from `root_actual_rows=9` to `rows_processed=19,803,951`.

`fix1_rows_processed.py` writes `tpch_dataset_corrected_v2.csv` (new file, `tpch_dataset_corrected.csv` untouched) — adds `rows_processed`, keeps the original `rows` column for comparison, and verifies `cost` / `time_ms` / `shared_hit` / `shared_read` / `rows` / `machine_id` are byte-identical to the base file.

Correlation with `scaling_factor_p50` (n=420 dev/prod pairs):

feature	pooled corr.	compute-labeled (n=60)	bandwidth (n=80)	io (n=80)	mixed (n=200)
old `rows`	−0.0132	−0.0504	**0.0085**	−0.0463	−0.0210
new `rows_processed`	0.0003	0.0446	**0.0113**	−0.0132	−0.0053

Verdict: the fix does not strengthen this correlation — both are essentially zero. This is not a failed fix so much as the wrong question answered correctly: `scaling_factor_p50 = prod_time / dev_time` is a ratio between two machines running the *same* query on the *same* data, so it is driven by hardware differences, not by how much work the query does. `rows` / `rows_processed` are per-query, hardware-invariant quantities (identical across all 5 machines for a given query), so correlating either against a cross-machine ratio pools different queries together and mostly measures noise. `rows_processed` is still the mechanically correct fix — it stops silently reporting near-zero "work done" for large aggregations — and it is the right input for a bottleneck *classifier* (within-query correctness across queries of different sizes matters there), which is exactly where Fix 2 uses it. Reported plainly: this specific correlation stays weak.

Fix 2: plan-structure features for the bottleneck classifier

`scripts/collection/extract_plan_features.py` additionally extracts, per query, hardware-invariant plan-structure features: `join_count` (Nested Loop/Hash Join/Merge Join node count), `node_count` (total plan nodes), `plan_depth`, `has_aggregate`, `has_sort`, and `correlated_pattern_count` (SubPlan/InitPlan nodes plus nodes with `Actual Loops > 1`, i.e. nested-loop-with-rescan or per-tuple subplan evaluation).

`fix2_classifier_plan_structure.py` re-runs the exact leave-one-query-out `RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=42)` from `phase5_classifier.py` /the buffer-bug correction, on `tpch_dataset_corrected_v2.csv`, for three feature sets:

feature set	macro-F1	compute F1	bandwidth F1	io F1	mixed F1
baseline (corrected buffers only, 6 features)	**0.375**	0.500	0.600	**0.000**	0.400
buffers + structure (13 features, as specified)	**0.336**	0.000	0.545	**0.000**	0.800
structure-only (7 features, diagnostic)	0.454	0.571	0.444	0.000	0.800

Baseline reproduces the prior report's 0.375 exactly, confirming the pipeline matches. **The classifier as specified (buffers + structure combined) does not improve — it gets worse (0.375 → 0.336), and io-class recall stays at exactly 0.000 in both.** Confusion matrices in `results/fix2_classifier_structural_summary.json`.

An honest side-finding: a diagnostic run using *only* the 7 structural features (no buffers at all) scores 0.454 — higher than both the baseline and the combined model. Plan structure does carry real signal on its own; concatenating 13 features onto a random forest trained on only 20 rows per leave-one-query-out fold appears to dilute rather than add, which is a small-sample effect, not evidence that structure is uninformative.

Honest check, as requested: the io class fails to get a single correct prediction in every configuration tried (baseline, combined, structure-only). Every io-labeled row's 5 machine-rows collapse to the same feature vector once buffers are corrected (io-bug fix) and structure is query-invariant, so a leave-one-query-out fold that has never seen an io-pattern query has nothing query-specific to generalize from — no feature set fixes that with only 4 held-out io queries (q4, q17, q20, q22) and 3 io-not-seen training queries at most per fold. **21 queries is the binding constraint here, not feature availability** — consistent with what the buffer-bug correction already found, and reconfirmed by a genuinely different feature family failing to move the needle.

Per spec, since the (buffers + structure) classifier did not improve, the headline scaling regression was **not** re-run in full. As a lighter-weight check: mean `p_compute` on the compute-labeled rows (q1/q12/q16, n=15) is 0.9922 under the baseline model and 0.9900 under the augmented model — unaffected. The headline multi-core-compute MAPE numbers (analytical 21.60%, gated 22.30%, significance CI [0.30, 2.55]) stand as previously reported; they were not disturbed by this fix. Model saved as `models/bottleneck_classifier_structural.pkl` (versioned alongside, not replacing, `models/bottleneck_classifier.pkl`), since it is not an improvement over the production model.

Fix 3: controlled parallelism experiment

`scripts/collection/parallelism_timing.py` ran all 21 queries on the local SF1 instance under `max_parallel_workers_per_gather = 0` (OFF) and `= 2` (ON, the collection default), 1 untimed warmup + 5 timed repeats per query per config (median of 5 used), via `EXPLAIN (ANALYZE, FORMAT JSON)`. Raw and median times: `results/fix3_parallel_timing.csv`.

`parallel_speedup = median(OFF) / median(ON)` was correlated against `benefit` (MAPE-reduction from switching to the multi-core compute signal, from `results/addition4b_per_query_benefit.csv`), across the same 21 queries:

statistic	value
Pearson r (n=21)	**0.458**, p=0.037
Spearman rho (n=21)	**0.503**, p=0.020
mean speedup, plan-parallel queries (n=19)	1.983x
mean speedup, plan-serial queries (n=2, q11/q20)	0.992x

Sanity check passes: q11 and q20, the two queries whose plans never use `Gather` / `Gather Merge` regardless of the setting, show ~1.0x speedup as expected — confirming the ON/OFF toggle is doing real work rather than measuring noise. The 5 queries with the largest multi-core benefit (q1 +47.4pp, q12 +40.2pp, q16 +32.6pp, q8 +23.7pp, q9 +22.1pp) all show speedups of 2.0x–2.8x; several queries with negative or ~0 multi-core benefit (q2 −25.1pp, q13 −0.02pp, q7 −8.4pp, q21 +5.9pp) show speedups below 1.1x or *slowdowns* under parallelism (q7 0.82x, q13 0.67x, q21 0.73x). Full per-query table: `results/fix3_parallelism_per_query.csv`.

Verdict: CONFIRMED (moderate, statistically significant). Enabling parallelism speeds up runtime, and the size of that speedup correlates positively and significantly ($p<0.05$ by both Pearson and Spearman) with how much a query benefited from the multi-core compute signal in the earlier cross-machine analysis. This is a direct causal test the static EXPLAIN-plan inspection (Addition 4b) could not provide — it only established *presence* of a parallel plan, not that parallelism execution actually drives the timing difference in the queries that need it. The correlation is moderate ($r\approx0.46$–0.50), not near-1.0, so this upgrades the mechanism from "leaning-confirmed" to "confirmed" without claiming it's the *only* factor at play.

Honest caveat: this experiment runs on one local machine, toggling a single GUC and measuring the resulting runtime change. It tests whether parallelism causes a speedup and whether that speedup tracks the queries where the multi-core signal mattered — it does not, and cannot, re-measure the original 5-EC2-machine cross-environment scaling result (those instances are terminated). It is a same-machine causal test of the mechanism, not a same-environment reproduction of the original finding. It strengthens the mechanism hypothesis; it does not replace the original cross-machine result.

Definition-of-done summary

fix	result
1: rows_processed	Mechanically correct fix (q1: 4 → 5.9M "rows"), but correlation with `scaling_factor_p50` stays ~0 for both old and new — legitimately weak, not improved, and explained (scaling factor is a cross-machine ratio; rows is a per-query constant).
2: plan-structure features	Combined classifier gets *worse* (0.375 → 0.336), io-recall stays 0.000 throughout. Structure-only diagnostic scores 0.454, showing the signal exists but doesn't combine well with buffers at n=21. Headline MAPE (21.60% / 22.30% / CI [0.30,2.55]) unaffected, confirmed still standing without a full re-run since the classifier didn't improve.
3: parallelism experiment	Pearson r=0.458 (p=0.037), Spearman rho=0.503 (p=0.020) — moderate, statistically significant positive correlation. Verdict upgraded from "leaning-confirmed" to **confirmed**, with the single-machine caveat stated plainly.

Artifacts

- `scripts/collection/extract_plan_features.py`, `results/plan_structure_features.csv`
- `fix1_rows_processed.py`, `tpch_dataset_corrected_v2.csv`, `results/fix1_rows_processed_correlation.csv`, `results/fix1_rows_processed_correlation_by_class.csv`
- `fix2_classifier_plan_structure.py`, `results/fix2_classifier_structural_summary.json`, `models/bottleneck_classifier_structural.pkl`, `models/bottleneck_classifier_structural_features.json`
- `scripts/collection/parallelism_timing.py`, `results/fix3_parallel_timing.csv`
- `fix3_parallelism_mechanism.py`, `results/fix3_parallelism_mechanism_summary.csv`, `results/fix3_parallelism_per_query.csv`